

在 Cloud Run 使用 vLLM 提供 Gemma 3 服務

來源：<https://codelabs.developers.google.com/devsite/codelabs/serve-gemma3-with-vllm-on-cloud-run?hl=zh-tw>

步驟數：9

本程式碼研究室會引導您在 Google Cloud Run 上運用 vLLM，部署及提供 Google 的 Gemma 3 模型。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 選擇合適的機型](#)
4. [4. 環境變數和密鑰](#)
5. [5. 建立服務帳戶](#)
6. [6. 在 Artifact Registry 建立映像檔](#)
7. [7. 測試服務](#)
8. [8. 結論](#)
9. [9. 清除](#)

1. 簡介

大型語言模型 (LLM) 正在改變我們建構智慧應用程式的方式。但要讓這些強大的模型準備好在現實世界中使用，可能並不容易。這類模型需要大量運算能力，尤其是顯示卡 (GPU)，以及能同時處理多項要求的智慧方式。此外，您希望降低成本，並確保應用程式順暢運作，不會發生延遲。

本程式碼研究室將說明如何解決這些難題！我們會使用兩項重要工具：

1. **vLLM**：您可以將這項技術視為 LLM 的超快速引擎。這能大幅提升模型執行效率，一次處理更多要求並減少記憶體用量。
2. **Google Cloud Run**：這是 Google 的無伺服器平台，這項服務非常適合部署應用程式，因為它會為您處理所有資源調度作業，從零位使用者到數千位使用者，再回到零位使用者，都能輕鬆應對。最重要的是，Cloud Run [現在支援 GPU](#)，這是託管 LLM 的必要條件！

結合使用 vLLM 和 Cloud Run，就能以強大、彈性且符合成本效益的方式提供 LLM 服務。在本指南中，您將部署開放模型，並以標準網頁 API 的形式提供。

課程內容

- 如何選擇適合放送的模型大小和變體。
- 如何設定 vLLM，以提供與 OpenAI 相容的 API 端點。
- 如何使用 Docker 將 vLLM 伺服器容器化。
- 如何將容器映像檔推送至 Google Artifact Registry。
- 如何將容器部署至 Cloud Run，並啟用 GPU 加速功能。
- 如何測試已部署的模型。

軟硬體需求

- 用來存取 Google Cloud 控制台的瀏覽器，例如 Chrome
 - 穩定的網路連線
 - 已啟用計費功能的 Google Cloud 專案
 - Hugging Face 存取權杖 (如果還沒有，請[在這裡](#)建立)
 - 對 Python、Docker 和指令列介面有基本的瞭解
 - 好奇心和學習熱忱
-

步驟 2 · 約 5 分鐘

2. 事前準備

設定 Google Cloud 專案

本程式碼研究室需要已啟用帳單帳戶的 Google Cloud 專案。

- **講師主導的課程：**如果您在教室中，講師會提供必要的專案和帳單資訊。按照指導老師的指示完成設定。
- **獨立學習者：**如果您是自行進行這項操作，且沒有現有的有效帳單帳戶，請使用自己的付款資訊設定帳單帳戶。請參閱 [Google Cloud Billing 說明文件](#)，建立新的帳單帳戶並為專案啟用。

建立 Google Cloud 專案

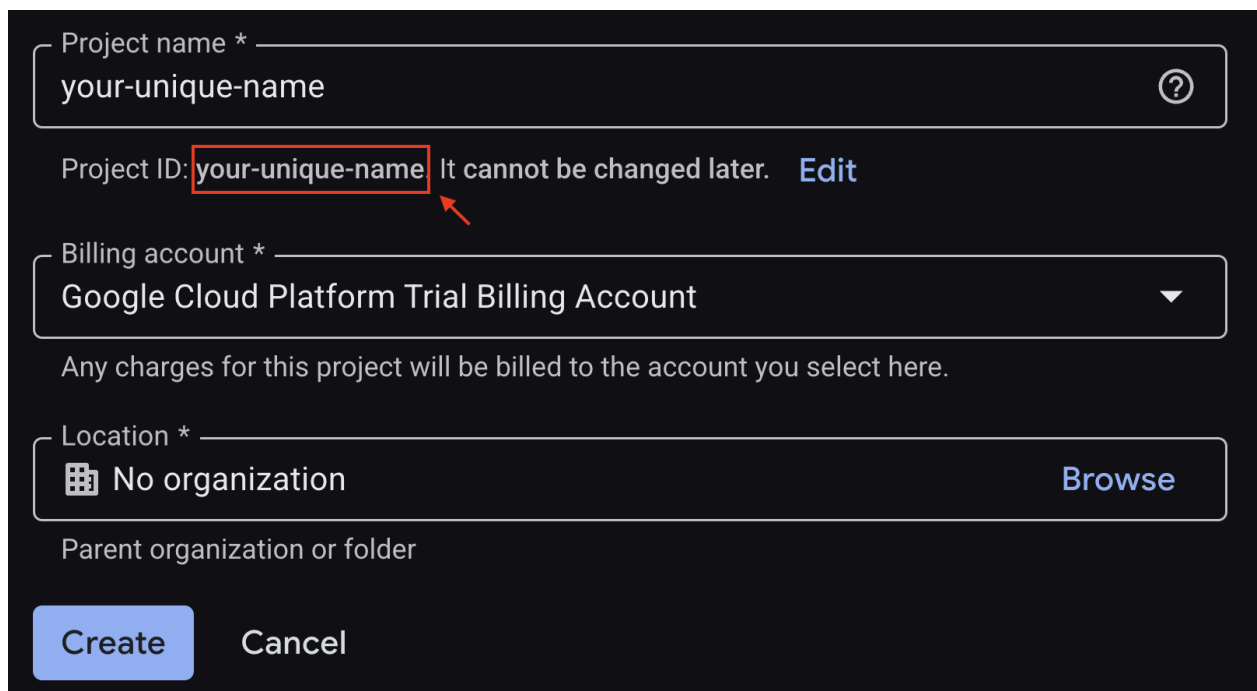
為確保本程式碼實驗室的所有工作井然有序，並與其他專案區隔開來，請先建立新的 Google Cloud 雲端專案。

如要開啟專案建立頁面，請按一下「Create New Project」（建立新專案）。

在專案建立頁面中輸入必要資訊：

- **專案名稱** - 您可以輸入任何想要的名稱 (例如 genai-workshop)
- **位置** - 保持「沒有機構」
- **帳單帳戶：**如果顯示這個選項，請選取「Google Cloud Platform 試用帳單帳戶」，或視需要選取自己的帳單帳戶。如果沒有看到這個選項，請繼續下一個步驟。

複製產生的專案 ID，後續步驟將會用到。

A screenshot of the Google Cloud Project creation form. The form is dark-themed with white text. It contains several input fields and a dropdown menu. The 'Project name' field is at the top, with the text 'your-unique-name' entered. Below it, the 'Project ID' is shown as 'your-unique-name', which is highlighted with a red box and a red arrow pointing to it. To the right of the Project ID, it says 'It cannot be changed later.' and 'Edit'. Below that is the 'Billing account' dropdown menu, which is set to 'Google Cloud Platform Trial Billing Account'. Underneath the billing account, it says 'Any charges for this project will be billed to the account you select here.' The 'Location' field is at the bottom, with the text 'No organization' and a 'Browse' button. At the very bottom, there are 'Create' and 'Cancel' buttons.

如果一切正常，請按一下「建立」按鈕。

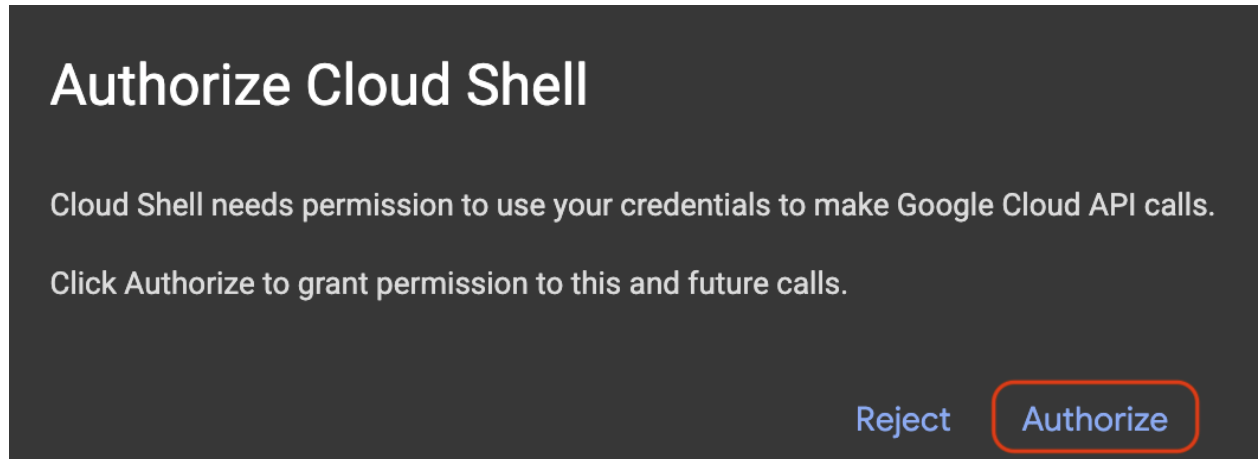
設定 Cloud Shell

Cloud Shell 是預先設定的環境，內含本程式碼研究室所需的所有工具。專案建立完成後，請按照下列步驟設定 **Cloud Shell**。

啟動 Cloud Shell

如要啟動 Cloud Shell，請按一下「啟動 Cloud Shell」。

如果出現要求授權的彈出式視窗，請點選「授權」。

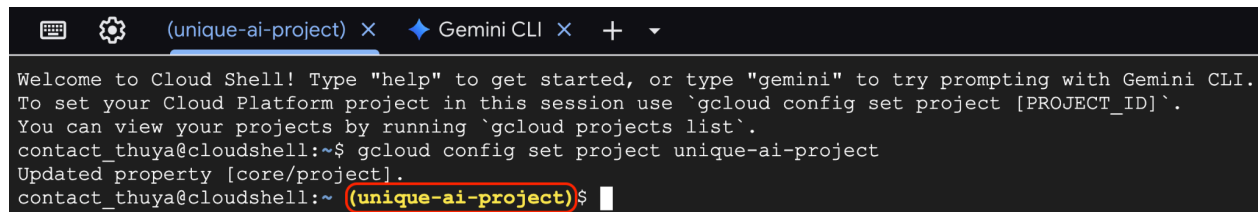


設定專案 ID

將 `replace-with-your-project-id` 替換為您在上述專案建立步驟中取得的實際專案 ID。在 Cloud Shell 終端機中執行下列指令，設定正確的專案 ID。

```
gcloud config set project replace-with-your-project-id
```

現在，您應該會在 Cloud Shell 終端機中看到已選取正確的專案。所選的專案 ID 會以黃色醒目顯示。



啟用必要的 API

如要使用 Cloud Run 等 Google Cloud 服務，請先為專案啟用相關 API。在 Cloud Shell 中執行下列指令，啟用本 Codelab 的必要服務：

```
gcloud services enable run.googleapis.com
gcloud services enable cloudbuild.googleapis.com
gcloud services enable secretmanager.googleapis.com
gcloud services enable artifactregistry.googleapis.com
```

3. 選擇合適的機型

您可以在 [Hugging Face Hub](#) 和 [Kaggle](#) 等網站上找到許多開放模型。如要在 Google Cloud Run 等服務上使用這些模型，您必須選擇符合現有資源 (即 NVIDIA L4 GPU) 的模型。

除了大小之外，請務必考量模型實際能執行的工作。模型各不相同，各有優缺點。舉例來說，有些模型可以處理不同類型的輸入內容 (例如圖片和文字，也就是多模態功能)，有些模型則可一次記憶及處理更多資訊 (也就是具有較大的脈絡視窗)。通常，較大型的模型會具備更進階的功能，例如[函式呼叫](#)和[思考](#)。

此外，也請務必確認服務工具 (在本例中為 vLLM) 是否支援所需模型。如要查看 vLLM 支援的所有模型，請按[這裡](#)。

現在，讓我們來探索 Gemma 3，這是 Google 最新推出的開放式大型語言模型 (LLM) 系列。Gemma 3 共有四種不同規模，複雜度以參數衡量：10 億、40 億、120 億和 270 億。

每種尺寸都有兩大類型：

- 基礎 (預先訓練) 版本：這是從大量資料中學習的基礎模型。
- 指令微調版本：這個版本經過進一步微調，可更瞭解及遵循特定指令或命令。

較大型的模型 (40 億、120 億和 270 億個參數) 屬於多模態模型，可解讀及處理圖片和文字。不過，最小的 10 億參數變體只專注於文字。

在本程式碼研究室中，我們將使用 10 億個變體的 Gemma 3：[gemma-3-1b-it](#)。使用較小的模型也有助於瞭解如何運用有限資源，這對於降低成本及確保應用程式在雲端順暢運作至關重要。

步驟 4 · 約 5 分鐘

4. 環境變數和密鑰

建立環境檔案

繼續操作前，建議您將本程式碼研究室中使用的所有設定集中管理。如要開始使用，請開啟終端機並按照下列步驟操作：

1. 為這項專案建立新資料夾。
2. 前往新建立的資料夾。
3. 在這個資料夾中建立空白的 `.env` 檔案 (這個檔案稍後會保存環境變數)

執行這些步驟的指令如下：

```
mkdir vllm-gemma3 && cd vllm-gemma3 && cloudshell edit .env
```

注意：如果您不習慣使用 `vim` 或 `nano` 等指令列工具，可以點選「開啟編輯器」按鈕，使用 **Cloud Code 編輯器**。

如果找不到 `.env` 檔案，請試著切換「編輯器」>「檢視」>「切換隱藏檔案」選項。

接著，複製下列變數，並貼到剛才建立的 `.env` 檔案中。

```
PROJECT_ID=your_project_id
REGION=your_region

MODEL_PROVIDER=google
MODEL_VARIANT=gemma-3-1b-it
MODEL_NAME=${MODEL_PROVIDER}/${MODEL_VARIANT}

AR_REPO_NAME=vllm-gemma3-repo
SERVICE_NAME=${MODEL_VARIANT}-service
IMAGE_NAME=${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO_NAME}/${SERVICE_NAME}

SERVICE_ACC_NAME=${SERVICE_NAME}-sa
SERVICE_ACC_EMAIL=${SERVICE_ACC_NAME}@${PROJECT_ID}.iam.gserviceaccount.com
```

請記得將預留位置值 (`your_project_id` 和 `your_region`) 替換為您的專案資訊。例如 `PROJECT_ID=unique-ai-project` 和 `REGION=europe-west4`。如要查看支援 Cloud Run GPU 的區域清單，請按這裡。

編輯並儲存 `.env` 檔案後，請輸入下列指令，將這些環境變數載入終端機工作階段：

```
source .env
```

您可以回應其中一個變數，測試變數是否已成功載入。例如：

```
echo $SERVICE_NAME
```

如果取得的值與您在 `.env` 檔案中指派的值相同，表示變數已成功載入。

在 Secret Manager 中儲存密鑰

對於任何私密資料 (包括存取代碼、憑證和密碼)，建議使用密鑰管理工具。

Gemma 3 模型設有使用限制，因此您必須先詳閱並接受條款及細則，才能使用。如要確認條款及細則，請參閱 [Hugging Face Hub 上的 Gemma 3 模型資訊卡](#)。

注意：如果還沒有 Hugging Face 存取權杖，請[在這裡](#)建立。後續步驟會用到這項資訊。

取得 Hugging Face 存取權杖後，請前往 Secret Manager 頁面，然後按照下列操作說明建立密鑰

- 前往 Google Cloud 控制台
- 從左上方的下拉式選單列選取專案
- 在搜尋列中搜尋「Secret Manager」，然後點選顯示的選項

在 Secret Manager 頁面中：

- 按一下「+Create Secret」(+建立密鑰) 按鈕，
- 填寫下列資訊：
- 名稱：HF_TOKEN
- 密鑰值：<your_hf_access_token>
- 完成後，按一下「建立密鑰」按鈕。

您現在應該已在 **Google Cloud Secret Manager** 中，將 Hugging Face 存取權杖設為密鑰。

您可以在終端機中執行下列指令，從 Secret Manager 擷取密鑰，測試是否能存取密鑰：

```
gcloud secrets versions access latest --secret=HF_TOKEN
```

終端機視窗中應該會顯示擷取到的存取權杖。

將 Secret 存取權授予 Cloud Build 服務帳戶

由於密鑰現在安全地儲存在 Secret Manager 中，

如要這麼做，請在終端機中執行下列指令：

```
# Get the project number
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')

# Grant Cloud Build access to the Hugging Face Token secret
gcloud secrets add-iam-policy-binding HF_TOKEN \
  --member="serviceAccount:${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com" \
  --role="roles/secretmanager.secretAccessor"
```


步驟 5 · 約 5 分鐘

5. 建立服務帳戶

為提升安全性，並在正式環境中有效管理存取權，服務應在專屬服務帳戶下運作，且嚴格限制為特定工作所需的權限。

執行下列指令來建立服務帳戶：

```
gcloud iam service-accounts create $SERVICE_ACC_NAME --display-name='Cloud Run vLLM Model  
Serving SA'
```

下列指令會附加必要權限

```
gcloud secrets add-iam-policy-binding HF_TOKEN \  
  --member="serviceAccount:${SERVICE_ACC_EMAIL}" \  
  --role="roles/secretmanager.secretAccessor"
```

6. 在 Artifact Registry 建立映像檔

這個步驟包括建立 Docker 映像檔，其中包含模型權重和預先安裝的 vLLM。

1. 在 Artifact Registry 建立 Docker 存放區

我們將在 Artifact Registry 中建立 Docker 存放區，用於推送建構的映像檔。在終端機中執行下列指令：

```
gcloud artifacts repositories create ${AR_REPO_NAME} \
  --repository-format docker \
  --location ${REGION}
```

2. 儲存模型

根據 [GPU 最佳做法文件](#)，您可以將機器學習模型儲存在容器映像檔中，或最佳化從 Cloud Storage 載入模型的方式。

當然，每種做法都有各自的優缺點。詳情請參閱說明文件。為求簡單起見，我們只會將模型儲存在容器映像檔。您會在下一個工作階段中執行這項操作。

3. 建立 Docker 檔案

建立名為 **Dockerfile** 的檔案，然後將下列內容複製到檔案中：

```
FROM vllm/vllm-openai:v0.16.0

ARG MODEL_NAME
ARG HF_TOKEN

ENV HF_HOME=/model-cache
ENV MODEL_NAME=${MODEL_NAME}

# Use the HF_TOKEN argument to log in and download the model
RUN huggingface-cli login --token ${HF_TOKEN} && \
    huggingface-cli download ${MODEL_NAME}

# Prevent vLLM from trying to download the model again at runtime
ENV HF_HUB_OFFLINE=1

EXPOSE 8080

ENTRYPOINT python3 -m vllm.entrypoints.openai.api_server \
  --port ${PORT:-8080} \
  --model ${MODEL_NAME} \
  --gpu-memory-utilization 0.90 \
  ${MAX_MODEL_LEN:+--max-model-len "$MAX_MODEL_LEN"}
```

4. 建立部署作業的 YAML 檔案

接著，在同一個目錄中建立名為 `cloudbuild.yaml` 的檔案。這個檔案定義 Cloud Build 要執行的步驟。複製下列內容並貼到 `cloudbuild.yaml`：

```

steps:
- name: 'gcr.io/cloud-builders/docker'
  entrypoint: 'bash'
  args:
  - '-c'
  - |
    docker build \
      --build-arg MODEL_NAME=${_MODEL_NAME} \
      --build-arg HF_TOKEN=${HF_TOKEN_SECRET} \
      -t ${_IMAGE_NAME} .
  secretEnv: ['HF_TOKEN_SECRET']

images:
- '${_IMAGE_NAME}'

availableSecrets:
  secretManager:
  - versionName: projects/${PROJECT_ID}/secrets/HF_TOKEN/versions/latest
    env: 'HF_TOKEN_SECRET'

```

5. 將建構提交至 Cloud Build

複製下列程式碼，並在終端機中運作執行：

```

gcloud builds submit . \
  --config=cloudbuild.yaml \
  --region=${REGION} \
  --substitutions=_MODEL_NAME=${MODEL_NAME},_IMAGE_NAME=${IMAGE_NAME}

```

這項指令會上傳程式碼 (`Dockerfile` 和 `cloudbuild.yaml`)、將殼層變數做為替代項目傳遞 (`_MODEL_NAME` 和 `_IMAGE_NAME`)，並啟動建構作業。

Cloud Build 現在會執行 `cloudbuild.yaml` 中定義的步驟。您可以在終端機中追蹤記錄，也可以點選 Cloud 控制台中的建構詳細資料連結。完成後，容器映像檔就會出現在 Artifact Registry 存放區中，隨時可供部署。

6. 部署至 Cloud Run

現在您可以將服務部署至 Cloud Run。在終端機中執行下列指令：

```

gcloud run deploy ${SERVICE_NAME} \
  --image ${IMAGE_NAME} \
  --region ${REGION} \
  --service-account ${SERVICE_ACC_EMAIL} \
  --gpu 1 \
  --gpu-type nvidia-l4 \
  --cpu 4 \
  --memory 16Gi \
  --max-instances 3 \
  --concurrency 80 \
  --no-allow-unauthenticated

```

注意：建構及推送映像檔的程序約需 **30 分鐘**，因為模型權重會直接整合至容器中。以這種方式嵌入權重，可避免啟動 Cloud Run 服務時通常會發生的冷啟動時間延長問題。

步驟 7 · 約 5 分鐘

7. 測試服務

在終端機中執行下列指令來建立 Proxy，以便在 localhost 中存取服務：

```
gcloud run services proxy ${SERVICE_NAME} --region ${REGION}
```

在新的終端機視窗中，於終端機執行 `curl` 指令，測試連線

```
curl -X POST http://localhost:8080/v1/completions \  
-H "Authorization: Bearer $(gcloud auth print-identity-token)" \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "google/gemma-3-1b-it",  
  "prompt": "Cloud Run is a ",  
  "max_tokens": 128,  
  "temperature": 0.90  
}'
```

如果看到類似下方的輸出內容：

```
{"id":"cmpl-  
e96d05d2893d42939c1780d44233defa","object":"text_completion","created":1746870778,"mo  
del":"google/gemma-3-1b-it","choices":[{"index":0,"text":"100% managed Kubernetes  
service. It's a great option for many use cases.\n\nHere's a breakdown of key  
features and considerations:\n\n* **Managed Kubernetes:** This means Google handles  
the underlying infrastructure, including scaling, patching, and maintenance. You  
don't need to worry about managing Kubernetes clusters.\n* **Serverless:** You only  
pay for the compute time your application actually uses. No charges when your code  
isn't running.\n* **Scalability:** Cloud Run automatically scales your application  
based on demand. You can easily scale up or down to handle fluctuating  
traffic.\n*","logprobs":null,"finish_reason":"length","stop_reason":null,"prompt_logp  
robs":null}], "usage":  
{"prompt_tokens":6,"total_tokens":134,"completion_tokens":128,"prompt_tokens_details"  
:null}}
```

8. 結論

恭喜！您已成功完成本程式碼研究室。您已學會如何：

- 為目標部署作業選擇適當的模型大小。
- 設定 vLLM，提供與 OpenAI 相容的 API。
- 使用 Docker 安全地將 vLLM 伺服器 and 模型權重容器化。
- 將容器映像檔推送至 Google Artifact Registry。
- 將 GPU 加速服務部署至 Cloud Run。
- 測試已部署的已驗證模型。

歡迎探索部署 Llama、Mistral 或 Qwen 等其他精彩模型，繼續學習之旅！

步驟 9 · 約 3 分鐘

9. 清除

為避免產生後續費用，請務必刪除您建立的資源。執行下列指令，清理專案。

1. 刪除 Cloud Run 服務：

```
gcloud run services delete ${SERVICE_NAME} --region=${REGION} --quiet
```

2. 刪除 Artifact Registry 存放區：

```
gcloud artifacts repositories delete ${AR_REPO_NAME} --location=${REGION} --quiet
```

3. 刪除服務帳戶：

```
gcloud iam service-accounts delete ${SERVICE_ACC_EMAIL} --quiet
```

4. 從 Secret Manager 刪除密鑰：

```
gcloud secrets delete HF_TOKEN --quiet
```
